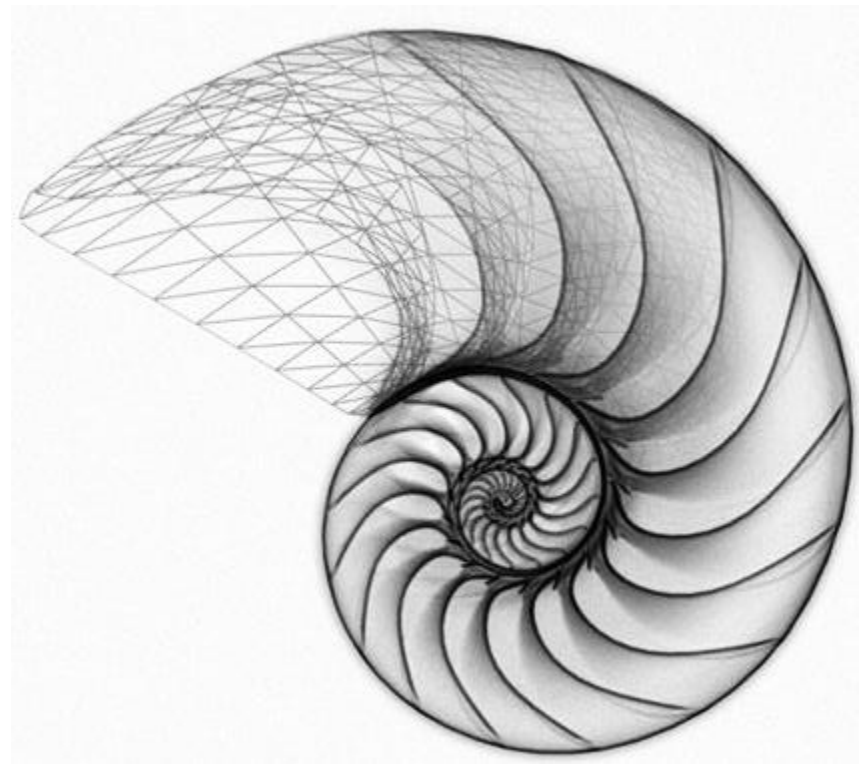




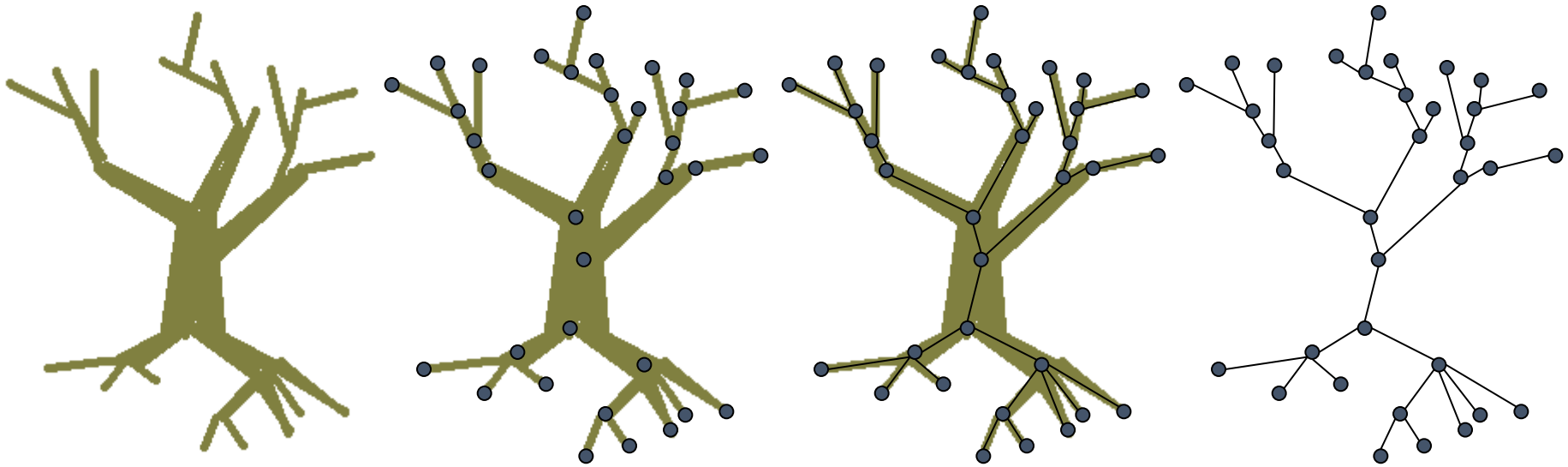
Lecture 15

Graphs

CSE225: Data Structures and Algorithms

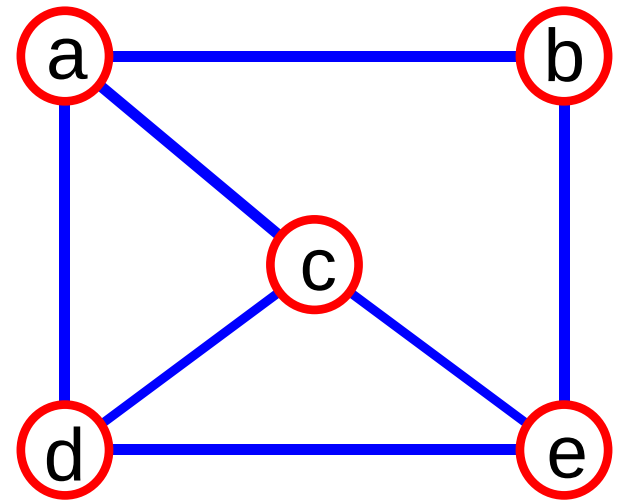
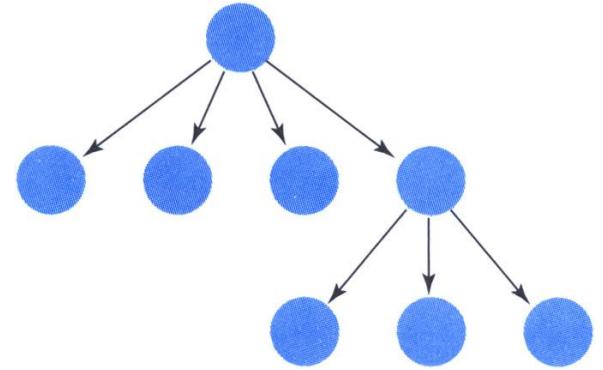
Graphs

- Recap: The tree data structure
 - Nodes
 - Parent-child relation between two nodes



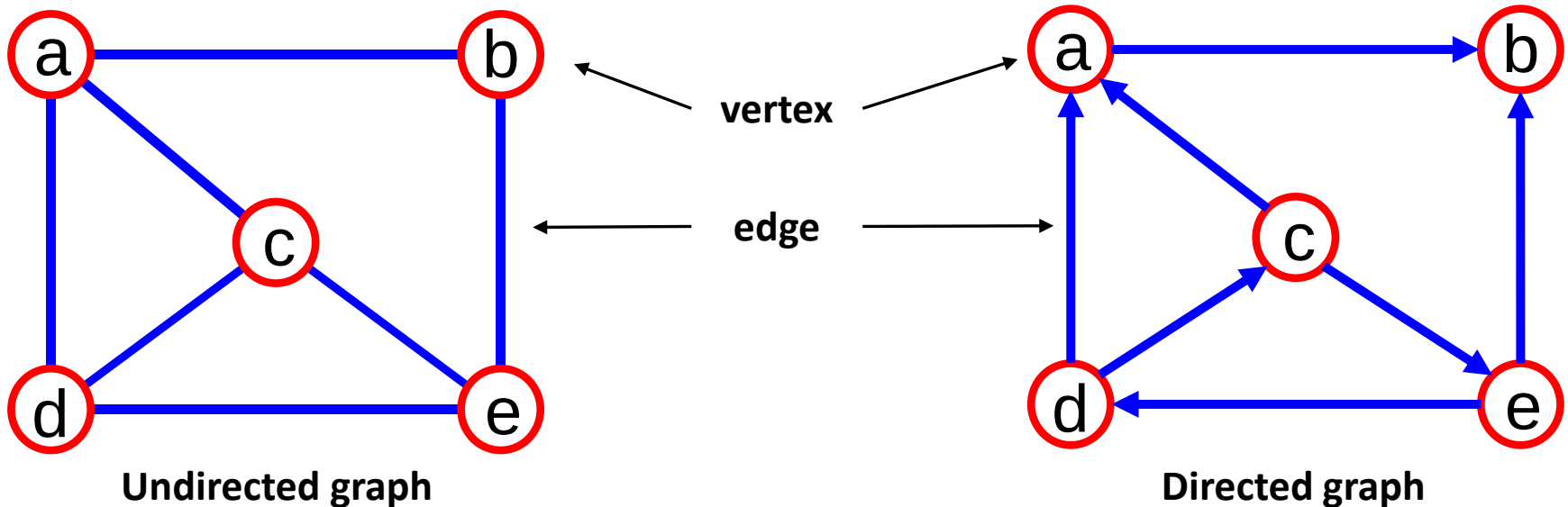
Graphs

- Recap: The tree data structure
 - Nodes
 - Parent-child relation between two nodes
- Remove the restriction that each node may have only one parent node
 - We have a graph



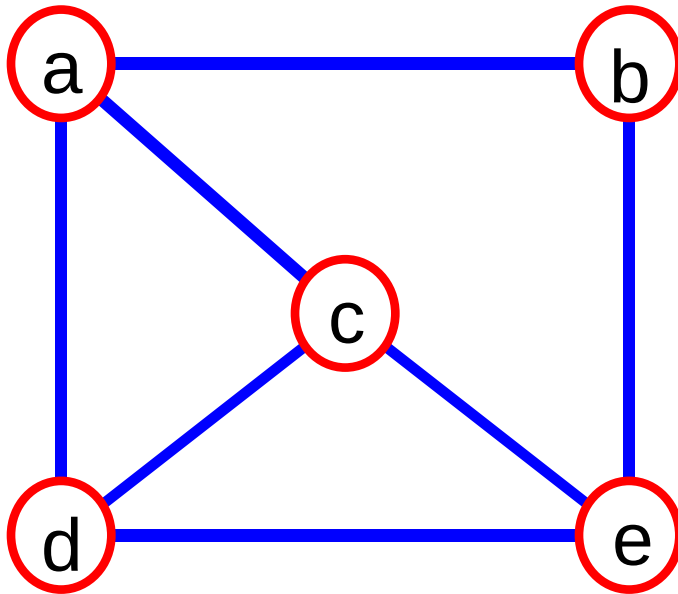
Graphs

- **Graph:** A data structure that consists of a set of nodes and a set of connections that relate the nodes to one another
- **Vertex:** A node in a graph
- **Edge (arc):** A pair of vertices representing a connection between two nodes in a graph
- **Undirected graph:** A graph in which the edges have no direction
- **Directed graph (digraph):** A graph in which each edge is directed from one vertex to another (or the same) vertex



Graphs

- A graph G is represented as $G = (V, E)$, where
 - V : set of **vertices**
 - E : set of **edges** connecting the **vertices** in V
- An **edge** $e = (u, v)$ is a pair of **vertices**
- Example:

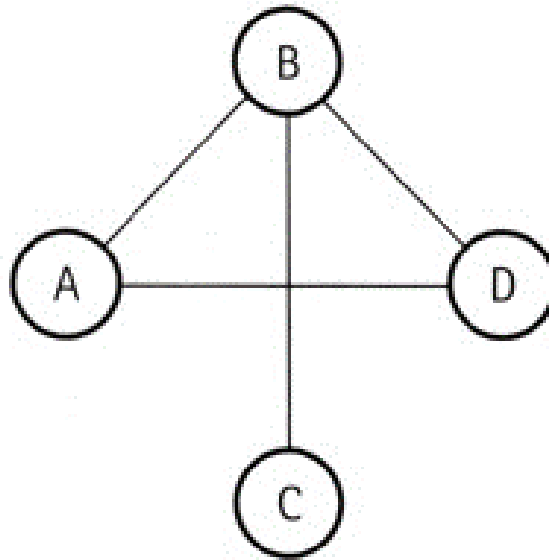


$$V = \{a, b, c, d, e\}$$

$$E = \{(a, b), (a, c), (a, d), (b, e), (c, d), (c, e), (d, e)\}$$

Graphs

- Example: Graph1

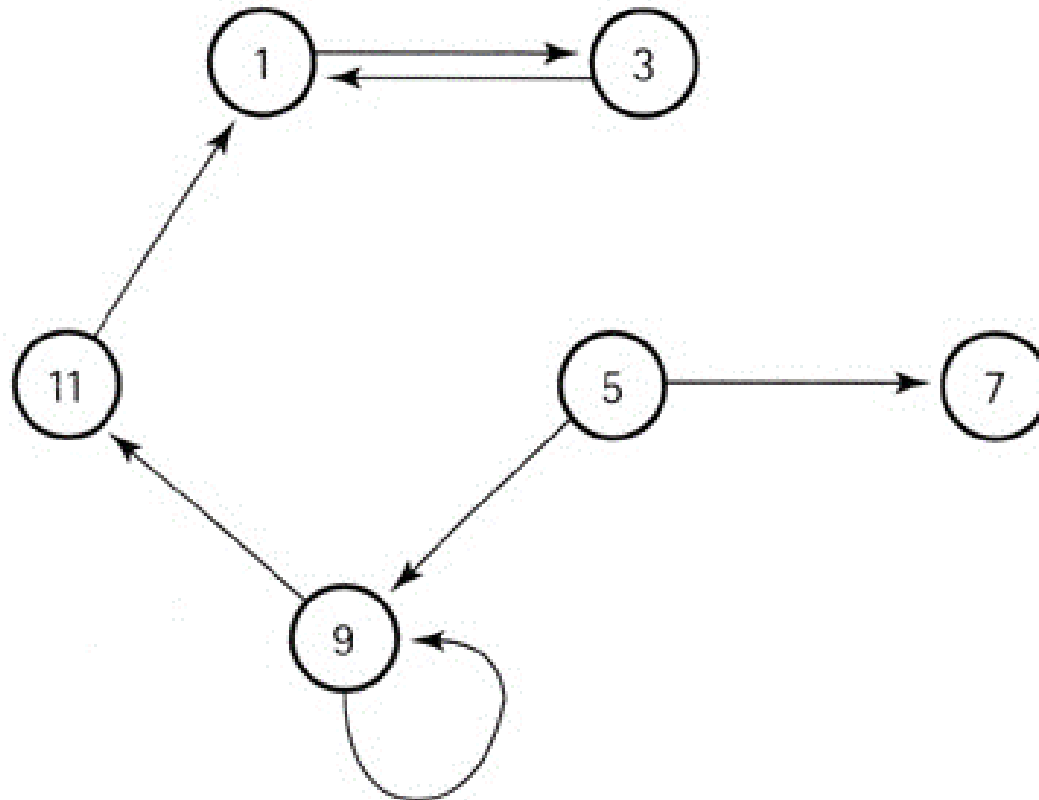


$V(\text{Graph1}) = \{A, B, C, D\}$

$E(\text{Graph1}) = \{(A, B), (A, D), (B, C), (B, D)\}$

Graphs

- Example: Graph2

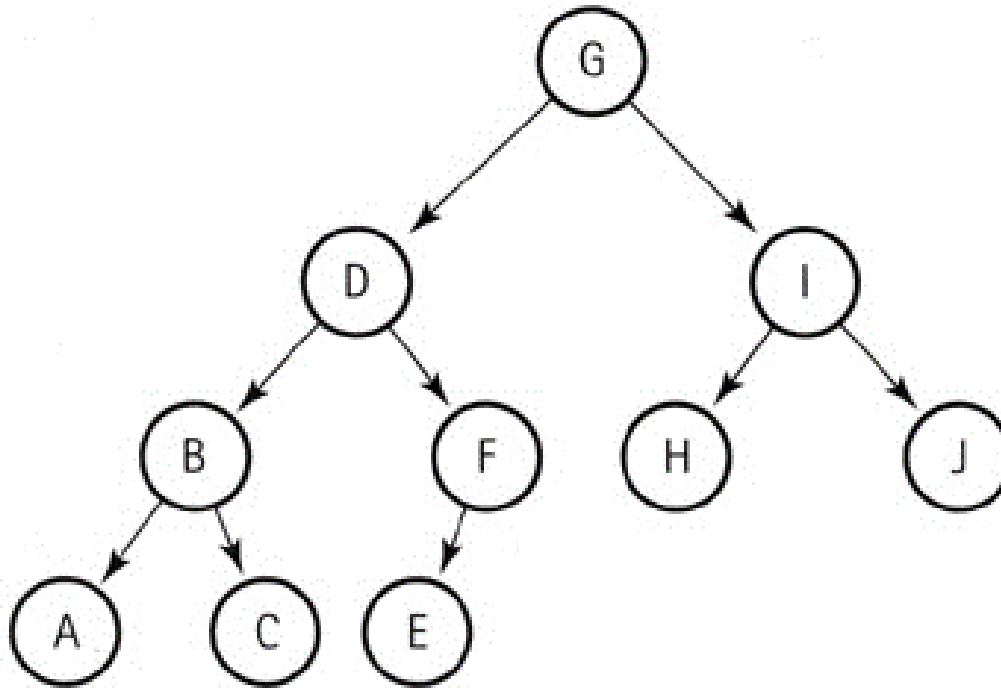


$V(\text{Graph2}) = \{1, 3, 5, 7, 9, 11\}$

$E(\text{Graph2}) = \{(1, 3), (3, 1), (5, 7), (5, 9), (9, 11), (9, 9), (11, 1)\}$

Graphs

- Example: Graph3

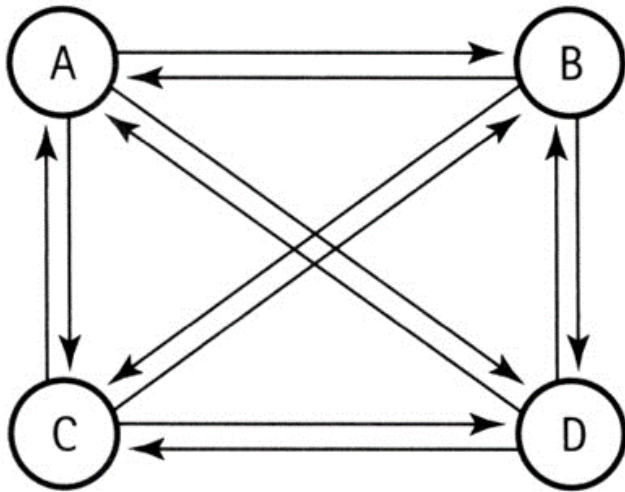


$V(\text{Graph3}) = \{A, B, C, D, E, F, G, H, I, J\}$

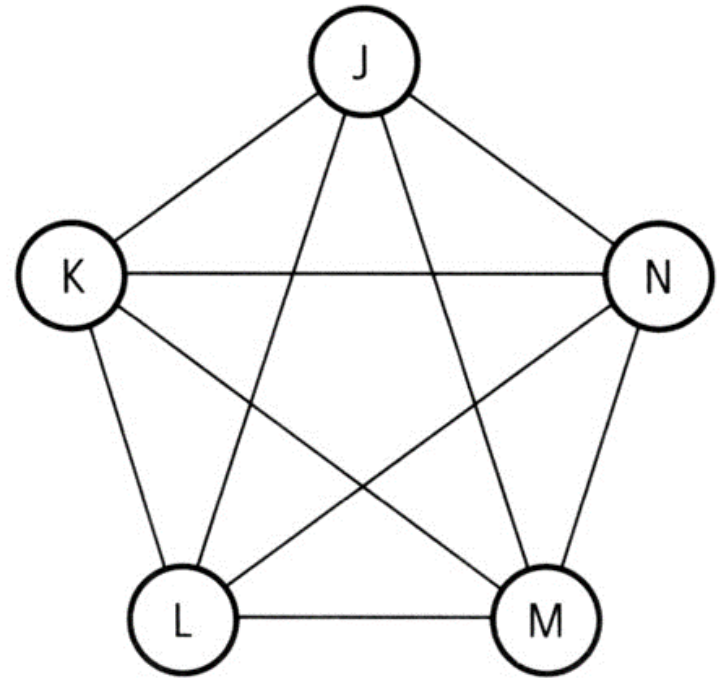
$E(\text{Graph3}) = \{(G, D), (G, I), (D, B), (D, F), (I, H), (I, J), (B, A), (B, C), (F, E)\}$

Graphs

- **Complete graph:** A graph in which every vertex is directly connected to every other vertex



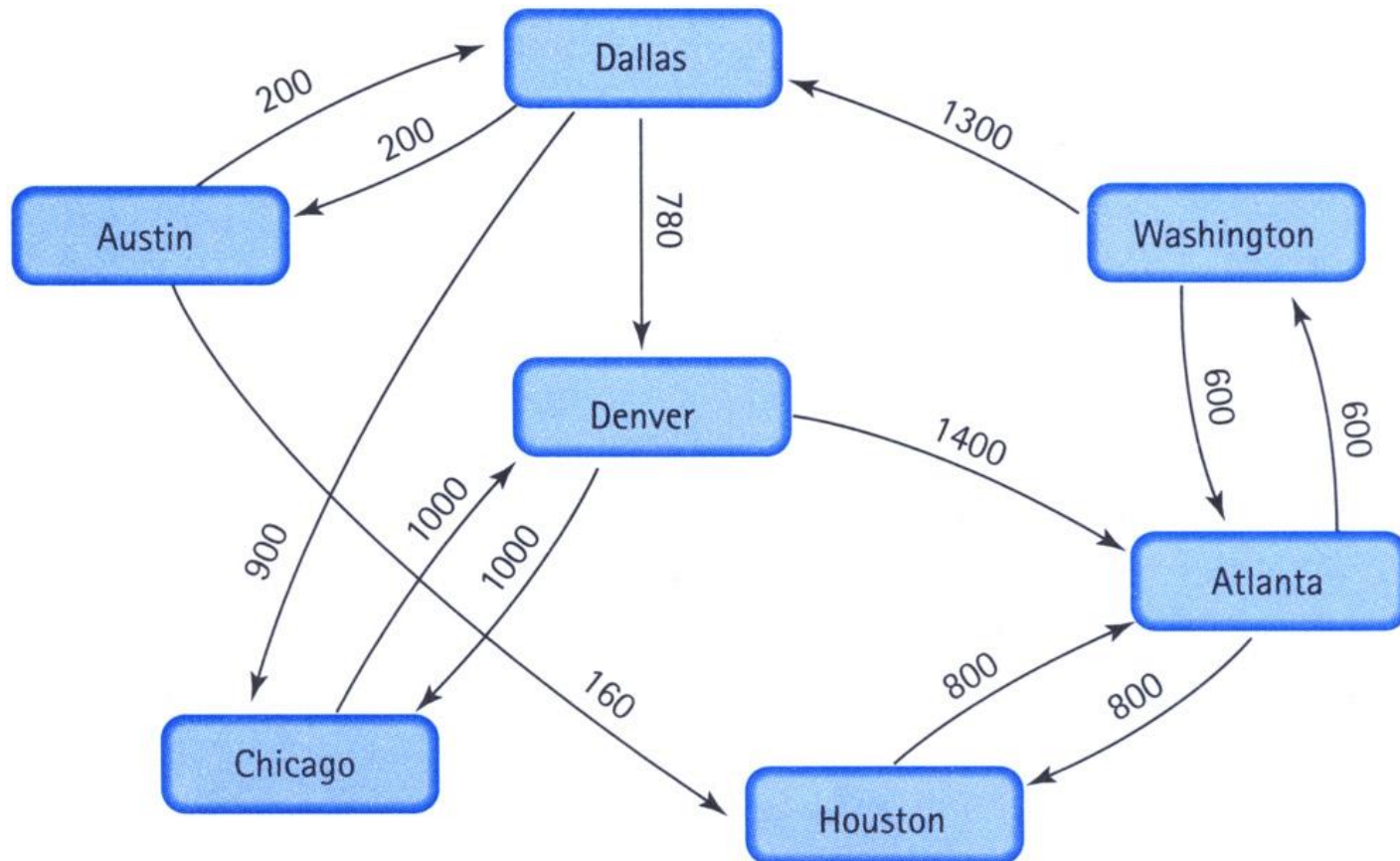
(a) Complete directed graph.



(b) Complete undirected graph.

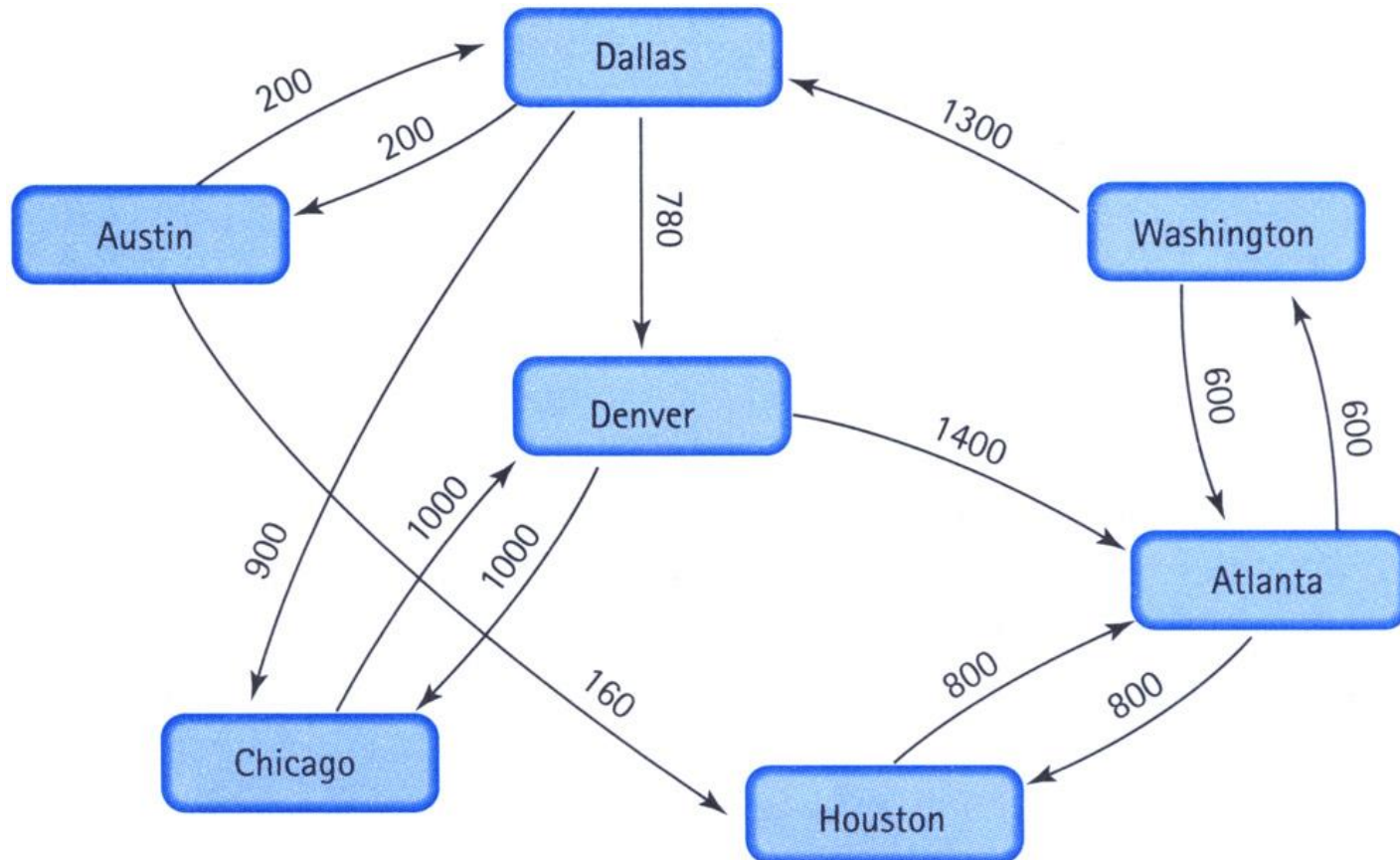
Graphs

- **Weighted graph:** A graph in which each edge carries a value (cost)



Graphs

- **Adjacent vertices:** Two vertices in a graph that are connected by an edge
- **Path:** A sequence of vertices that connects two nodes in a graph



Implementation Issues

- How can we store a graph in memory?

0	Atlanta
1	Austin
2	Chicago
3	Dallas
4	Denver
5	Houston
6	Washington
7	
8	
9	

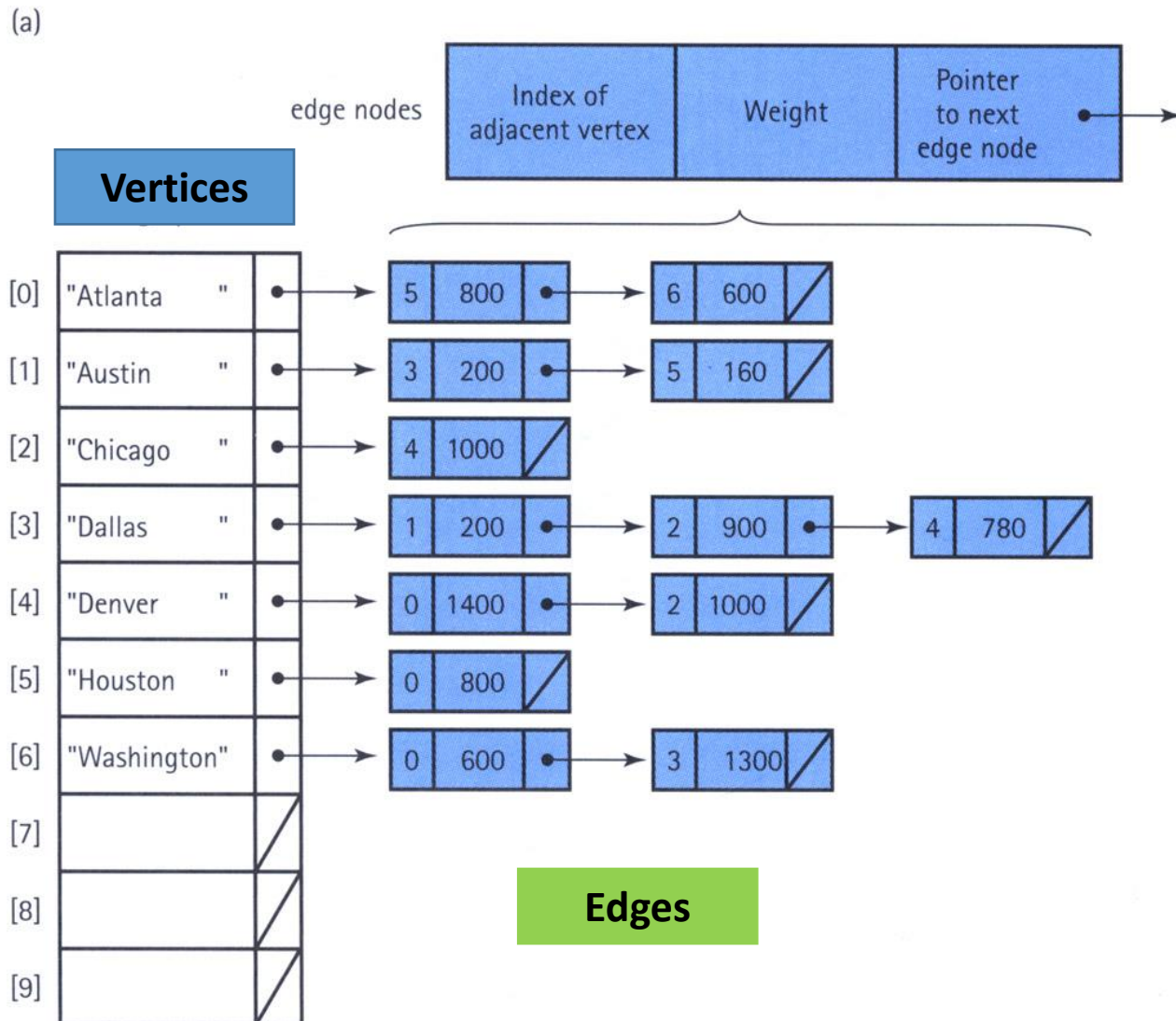
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
[0]	0	0	0	0	0	800	600			
[1]	0	0	0	200	0	160	0			
[2]	0	0	0	0	1000	0	0			
[3]	0	200	900	0	780	0	0			
[4]	1400	0	1000	0	0	0	0			
[5]	800	0	0	0	0	0	0			
[6]	600	0	0	1300	0	0	0			
[7]										
[8]										
[9]										

Vertices

Edges

Implementation Issues

- How can we store a graph in memory?



Graph Specification

Structure:	The graph consists of a set of vertices and a set of weighted edges that connect some or all of the vertices to one another.
Operations:	
MakeEmpty	
Function	Initializes the graph to an empty state.
Postcondition	Graph is empty.
Boolean IsEmpty	
Function	Tests whether the graph is empty.
Postcondition	Returns true if graph is empty and false otherwise.
Boolean IsFull	
Function	Tests whether the graph is full.
Postcondition	Returns true if graph is full and false otherwise.

Graph Specification

AddVertex(VertexType vertex)

Function	Adds vertex to the graph.
----------	---------------------------

Precondition	Graph is not full.
--------------	--------------------

Postcondition	vertex is in $V(\text{graph})$.
---------------	----------------------------------

AddEdge(VertexType fromVertex, VertexType toVertex, EdgeValueType weight)

Function	Adds an edge with the specified weight from fromVertex to toVertex.
----------	---

Precondition	fromVertex and toVertex are in $V(\text{graph})$.
--------------	--

Postcondition	$(\text{fromVertex}, \text{toVertex})$ is in $E(\text{graph})$ with the specified weight.
---------------	---

Graph Specification

EdgeValueType Weights(VertexType fromVertex, VertexType toVertex)	
Function	Determines the weight of the edge from fromVertex to toVertex.
Precondition	fromVertex and toVertex are in V(graph).
Postcondition	Function value = weight of edge from fromVertex to toVertex, if edge exists. If edge does not exist, function value = special "null-edge" value.
GetToVertices(VertexType vertex, QueType& vertexQ)	
Function	Returns a queue of the vertices that are adjacent from vertex.
Precondition	vertex is in V(graph).
Postcondition	vertexQ contains the names of all vertices that are adjacent from vertex.

graphtype.h

```
#include "QueType.h"
```

```
template<class VertexType>
```

```
class GraphType
```

```
{
```

```
public:
```

```
    GraphType();
```

```
    GraphType(int maxV);
```

```
    ~GraphType();
```

```
    void MakeEmpty();
```

```
    bool IsEmpty() const;
```

```
    bool IsFull() const;
```

```
    void AddVertex(VertexType);
```

```
    void AddEdge(VertexType, VertexType, int);
```

```
    int WeightIs(VertexType, VertexType);
```

```
    void GetToVertices(VertexType, QueType<VertexType>&);
```

```
    void ClearMarks();
```

```
    void MarkVertex(VertexType);
```

```
    bool IsMarked(VertexType);
```

```
private:
```

```
    int numVertices;
```

```
    int maxVertices;
```

```
    VertexType* vertices;
```

```
    int **edges;
```

```
    bool* marks;
```

```
};
```

graphtype.cpp

```
#include "GraphType.h"
template<class VertexType>
GraphType<VertexType>::GraphType()
{
    numVertices = 0;
    maxVertices = 50;
    vertices = new VertexType[50];
    edges = new int*[50];
    for(int i=0;i<50;i++)
        edges[i] = new int [50];
    marks = new bool[50];
}
template<class VertexType>
GraphType<VertexType>::GraphType(int maxV)
{
    numVertices = 0;
    maxVertices = maxV;
    vertices = new VertexType[maxV];
    edges = new int*[maxV];
    for(int i=0;i<maxV;i++)
        edges[i] = new int [maxV];
    marks = new bool[maxV];
}
```

graphtype.cpp

```
template<class VertexType>
GraphType<VertexType>::~~GraphType()
{
    delete [] vertices;
    delete [] marks;
    for(int i=0;i<maxV;i++)
        delete [] edges[i];
    delete [] edges;
}

const int NULL_EDGE = 0;
template<class VertexType>
void GraphType<VertexType>::AddVertex(VertexType vertex)
{
    vertices[numVertices] = vertex;

    for (int index = 0; index <= numVertices; index++)
    {
        edges[numVertices][index] = NULL_EDGE;
        edges[index][numVertices] = NULL_EDGE;
    }
    numVertices++;
}
```

graphtype.cpp

```
template<class VertexType>
int IndexIs(VertexType* vertices, VertexType vertex)
{
    int index = 0;

    while (!(vertex == vertices[index]))
        index++;
    return index;
}
```

```
template<class VertexType>
void GraphType<VertexType>::AddEdge(VertexType fromVertex,
    VertexType toVertex, int weight)
{
    int row = IndexIs(vertices, fromVertex);
    int col = IndexIs(vertices, toVertex);
    edges[row][col] = weight;
}
```

graphtype.cpp

```
template<class VertexType>
int GraphType<VertexType>::WeightIs(VertexType fromVertex, VertexType
toVertex)
{
    int row = IndexIs(vertices, fromVertex);
    int col= IndexIs(vertices, toVertex);

    return edges[row][col];
}
```

```
template<class VertexType>
void GraphType<VertexType>::GetToVertices(VertexType vertex,
QueType<VertexType>& adjVertices)
{
    int fromIndex;
    int toIndex;

    fromIndex = IndexIs(vertices, vertex);
    for (toIndex = 0; toIndex < numVertices; toIndex++)
        if (edges[fromIndex][toIndex] != NULL_EDGE)
            adjVertices.Enqueue(vertices[toIndex]);
}
```